

Project Report: ApexFlow

Development of a Telemetry and Analysis SaaS for SimRacing

1. Introduction and Motivation

Competitive motorsport is a sport where milliseconds make the difference between victory and defeat. In both real life and high-level simulators (SimRacing), a driver's most powerful tool is not the steering wheel, but the data. However, access to professional telemetry software is often limited to engineers, requiring advanced technical knowledge to interpret endless spreadsheets or unintuitive graphs.

The primary motivation behind **ApexFlow** was born from a personal need: to democratize access to telemetry. My goal was to build a system that not only captured real-time data from a simulator (Assetto Corsa) but also processed and presented it in a visual, engaging, and understandable way through a modern web architecture.

This project is not just a simple visualization dashboard; it is a complete "End-to-End" architecture covering everything from binary memory extraction (Edge/IoT) to database persistence and analytical visualization using mathematical comparison algorithms.

2. System Architecture

To ensure the system was scalable and had low latency, I decided to move away from traditional monolithic applications and design a highly decoupled architecture. The project is divided into three main layers:

- **Extraction Layer (Local Agent):** A Python script responsible for intercepting the racing simulator's network communications.
- **Processing Layer (Backend/Cloud):** A server developed with FastAPI that manages simultaneous connections, broadcasts live data, and stores historical sessions.
- **Presentation Layer (Frontend):** A reactive and asynchronous web interface for displaying the instrument panel and performing comparative lap analysis.

This separation ensures that, in the future, the backend and frontend can be hosted on a cloud server (AWS, Render, Railway), while the local agent continues to run on the user's PC, effectively turning ApexFlow into a true SaaS (Software as a Service) product.

3. Data Ingestion: The IoT Agent and the Binary Barrier

The first major technical challenge was data acquisition. Professional simulators like Assetto Corsa do not export their telemetry in readable formats like JSON or XML due to extreme performance constraints. Instead, they broadcast compressed memory blocks in binary code (C++) via the UDP protocol.

To solve this, I developed agent.py. The workflow for this agent is as follows:

1. **Handshake:** It opens a UDP socket and communicates with the simulator's local port 9996, sending a formal subscription request to initiate the transmission.
2. **Binary Unpacking:** Using Python's native struct library, the agent maps and translates the received byte stream. I had to research the RTCarInfo data structure of the game engine to accurately extract variables such as speed (float), RPM (float), and pedal inputs (gas, brake).
3. **Transmission:** The unpacked data is formatted and sent at a rate of 60Hz via WebSockets to the central server.

4. Backend: Asynchronous Processing and Persistence

The core of the application is cloud_backend.py. I chose **FastAPI** for its excellent handling of asynchronous operations and its native performance.

Real-Time Communication (WebSockets)

For a racing car's instrument panel, the traditional HTTP protocol (based on request-response) was unviable; constant polling would overload the server and introduce severe lag. I implemented WebSockets to keep a bidirectional tunnel open. The server receives telemetry from the Agent and instantly broadcasts it to all connected web clients.

Efficient Storage (SQLite)

Saving every data packet received at 60 Hertz would destroy any database's I/O performance in minutes if done row by row. To optimize this, I designed a batching system. The backend accumulates data in RAM and, at specific intervals, performs a single massive bulk insert into a **SQLite** database. This ensures that real-time performance is not penalized by disk writing operations.

5. The Mathematical Challenge: Trace Synchronization (Ghost Lap)

One of the key features I wanted to implement was the "Analysis Mode": the ability to superimpose the graph of a current lap with a reference lap (the professional driver or *Ghost Lap*). Here, I faced the most complex technical challenge of the project.

Initially, charting libraries stacked the data based on the temporal array index (e.g., data point #500 from Lap 1 was compared with data point #500 from Lap 2). This created a severe issue: "Outlaps" (laps exiting the pits). If the car was stationary in the pitlane for 10 seconds, thousands of data points were generated with a speed of "0 km/h". When overlaying the graphs, the braking and acceleration curves were completely misaligned, rendering the tool useless.

The Solution: I decided to abandon time as a measurement standard and pivot to a geospatial metric. I modified the agent to calculate the exact distance traveled using the rectilinear motion formula ($\text{Distance} = \text{Speed} \times \text{Time elapsed between frames}$).

On the frontend, I rewrote the chart rendering to use a **linear X-axis**. I transformed the data lists into strict Cartesian coordinates (X: Meters on track, Y: Speed). Thanks to this, it doesn't matter if the car was stationary for half an hour at the starting line; meter 450 of the track will always perfectly align with meter 450 of the reference lap.

6. UI/UX Design and High-Performance Visualization

The user interface could not be a simple table of numbers. I wanted to replicate the high-pressure, high-tech environment of a Formula 1 pit wall.

I used **Tailwind CSS** to build a responsive and modern interface. I adopted a *Glassmorphism* design language (translucent dark panels, background blurs, and subtle illuminated borders) to reduce visual fatigue.

- **Live Dashboard:** Features digital LCD displays, vertical progress bars for pedal telemetry, and a dynamic color gradient for the RPM bar simulating the rev limiter.
- **Graphics Engine:** I integrated **Chart.js**, disabling real-time animations and optimizing axis rendering so the browser could draw hundreds of points per second without dropping frames (FPS).

7. Innovation: The Heuristic Track Engineer

I didn't want the project to remain just a graph-drawing tool; I wanted it to provide real value to the user. For this reason, I developed the "Smart Coach".

I implemented a heuristic algorithm on the server that executes when a user requests a lap comparison. The system cross-references the data dictionaries of both laps, looking for mathematical discrepancies above a defined threshold. If it detects that at a specific meter on the track the user's speed is 20 km/h lower than the reference lap, the algorithm evaluates the context:

- If the overall speed is low, it infers that it is the apex of a corner and outputs: *"You are losing speed mid-corner. Try letting the car roll more."*
- If the discrepancy occurs in a high-speed deceleration zone, it infers a braking error and outputs: *"You are braking too early compared to the reference."*

This natural language processing layer adds massive differential value to the product, bringing it closer to enterprise-level solutions.

8. Conclusions

Developing ApexFlow has been an exhaustive journey through multiple software engineering disciplines. It required understanding low-level network protocols (UDP), mastering concurrency in Python, designing database schemas for high-frequency time series, solving mathematical data alignment problems, and building an attractive, reactive user interface.